POLICYLENS

RBI Document Ingestion and Semantic Retrieval System

Project Report

Submitted by: Jitesh Vishwakarma

Institution: ITM Skills University

Programme / Course: Agentic AI

Abstract

PolicyLens is a document-ingestion and semantic-retrieval system designed around a corpus of 30 public Reserve Bank of India (RBI) PDF documents. The corpus includes text-native, scanned and table-heavy documents. The system prepares these documents for retrieval by extracting text, applying an OCR fallback for scanned material, cleaning extracted text, creating chunks, enriching chunks with metadata, generating vector embeddings and storing the resulting records in ChromaDB. The embedding model used in the final experiments is **intfloat/e5-base-v2**.

Two chunking approaches were evaluated: fixed-size chunking and structure-aware chunking. The final structure-aware collection contains 15,374 chunks, while the fixed-size collection contains 19,341 chunks. Retrieval was evaluated with 25 questions: 15 standard questions, five paraphrased questions and five table-based questions. The structure-aware configuration achieved Recall@3 of 0.88, Recall@5 of 0.96, Recall@10 of 1.00 and MRR of 0.7743. The fixed-size configuration achieved 0.72, 0.84, 0.92 and 0.7613 respectively.

On this evaluation set, structure-aware chunking performed better on all four aggregate metrics, although fixed-size chunking retrieved the relevant result more effectively for some individual questions, including certain table-based questions. PolicyLens is a retrieval system: it returns ranked relevant passages and metadata. LLM-based answer generation is outside the scope of this implementation.

1. Introduction

Regulatory documents contain detailed rules, definitions, conditions, numerical thresholds and tables. Finding a relevant passage in a large collection of PDFs can be time-consuming, particularly when the user describes a rule in different words from the source document. PolicyLens addresses this retrieval problem by converting RBI PDFs into searchable text chunks and retrieving passages based on semantic similarity, with metadata filters available to narrow the search.

1.1 Problem statement

A collection of regulatory PDFs may contain different document layouts and content types. Text-native PDFs can usually be processed through text extraction, while scanned PDFs may require OCR. Tables and section boundaries can also be difficult to represent as generic text. A retrieval pipeline therefore needs a consistent ingestion process, useful metadata, and a chunking strategy that preserves enough context for search.

1.2 Project objectives

The objectives of PolicyLens are to: (1) ingest a corpus of 30 public RBI PDFs; (2) extract text and provide OCR fallback for scanned documents; (3) clean and normalize the extracted text; (4) create fixed-size and structure-aware chunks; (5) attach document and page metadata; (6) generate embeddings and store them in ChromaDB; (7) support semantic search and metadata filtering; and (8) compare retrieval performance using a 25-question evaluation dataset.

1.3 Scope and limitations

The implemented system focuses on ingestion, vector indexing and retrieval. It does not generate natural-language answers with an LLM. Evaluation measures whether relevant document/page passages are retrieved and how highly they are ranked; it does not measure the factual correctness of generated answers. Results are based on the defined 25-question test set and should not be interpreted as proof of performance on every possible RBI query.

2. Dataset and document inventory

The corpus contains 30 PDF files registered in **data/manifest.csv**. The corpus audit verified 30 manifest records and 30 corresponding PDFs. It found no duplicate document IDs or filenames, missing PDFs, page-count mismatches or unregistered PDFs.

Document category	Count
Text-native	19
Scanned	6
Table-heavy	5
Total	30

The manifest provides document-level information used during ingestion and retrieval, including identifiers and descriptive metadata. Each stored chunk also carries metadata such as document ID, title, issuer, issue date, page range, section title and source hash, where available.

3. System architecture and methodology

The processing pipeline begins with the PDF corpus and manifest. The ingestion process extracts text using PyMuPDF and uses OCR as a fallback for scanned pages. Extracted text is cleaned and normalized before chunking. Each chunk is enriched with metadata, converted into an embedding using intfloat/e5-base-v2 and stored in a ChromaDB collection.

3.1 Processing stages

Stage	Purpose
PDF ingestion	Loads the 30 source PDFs and reads their manifest records.
Text extraction	Uses PyMuPDF to extract text from text-native PDF pages.
OCR fallback	Provides a text-recognition path for scanned pages.
Text cleaning	Normalizes extracted text before chunk creation.
Chunking	Creates fixed-size or structure-aware text units.
Metadata enrichment	Associates chunks with document identity, page and other available metadata.
Embedding	Uses intfloat/e5-base-v2 to convert chunks into vectors.
Vector storage	Stores vectors, text and metadata in ChromaDB.
Retrieval	Embeds a query, searches the collection and returns ranked passages.

3.2 Chunking strategies

Fixed-size chunking: text is divided into chunks with a maximum of 300 tokens and 60-token overlap. The fixed-size collection contains 19,341 chunks, with an average of 299.8 tokens and a maximum of 300 tokens.

Structure-aware chunking: the pipeline uses detected document structure and section boundaries to form chunks, splitting long sections into chunks of up to approximately 300 tokens. The final structure-aware collection contains 15,374 chunks. This approach aims to keep related content together while avoiding oversized chunks.

Both final collections were built from the same 30-document corpus and used the same embedding model. Their chunking rules differ, so the experiment compares the overall retrieval outcomes of the two implemented approaches.

4. Retrieval and evaluation design

At query time, PolicyLens generates a query embedding and searches the selected ChromaDB collection for semantically similar chunks. Results are returned as ranked passages with document and page metadata. Metadata filters can restrict retrieval using fields such as document ID or issuer. The command-line interface supports a configurable top-k value and metadata filter expressions.

4.1 Evaluation dataset

The evaluation dataset contains 25 questions with expected answers and gold document/page locations. It contains 15 standard questions (Q01–Q15), five paraphrased questions (Q16–Q20) and five table-based questions (Q21–Q25). The paraphrased questions test whether retrieval works when the question wording differs from the source. The table-based questions ask for values or entries from regulatory tables and illustrations.

4.2 Metrics

Recall@k measures the proportion of evaluation questions for which a relevant document/page result appears within the first k retrieved results. **MRR (Mean Reciprocal Rank)** averages the reciprocal of the rank of the first relevant result across questions. A higher MRR indicates that relevant results tend to appear closer to the top of the ranking.

4.3 Experimental controls

The comparison uses the same 30 PDFs, the same 25 evaluation questions and the same embedding model. Separate ChromaDB collections and separate CSV output files were used for the two strategies, preserving the structure-aware evaluation outputs while evaluating the fixed-size collection.

The evaluation script records question ID, rank, retrieved document ID, title, page range, similarity score and whether the retrieved result matches the gold document/page annotation. The per-question retrieval logs provide the basis for the aggregate metrics and strategy comparison.

5. Results

Metric	Structure-aware	Fixed-size
Recall@3	0.8800 (88%)	0.7200 (72%)
Recall@5	0.9600 (96%)	0.8400 (84%)
Recall@10	1.0000 (100%)	0.9200 (92%)
MRR	0.7743	0.7613

Structure-aware chunking achieved higher Recall@3, Recall@5 and Recall@10. It also achieved a slightly higher MRR. The fixed-size collection contained more chunks (19,341 versus 15,374), but a larger number of chunks did not translate into higher aggregate retrieval performance on this test set.

6. Chunking strategy comparison

The retrieval logs show that the strategies do not produce identical rankings for every question. The following cases illustrate differences in the top-ranked result. Correctness here refers to the evaluation script's gold document/page matching, not to a generated answer.

Question	Structure-aware top result	Fixed-size top result
Q04 – Digital lending data collection	RBI_CREDIT_001, p. 23; correct	RBI_CREDIT_001, pp. 18–19; not correct
Q07 – Project finance disbursement	RBI_CREDIT_001, pp. 72–73; not correct	RBI_CREDIT_001, p. 47; correct
Q12 – Board-approved credit policy	RBI_CREDIT_001, pp. 14–16; correct	RBI_CREDIT_001, p. 89; not correct

6.1 Interpretation

Q04 and Q12 show cases where the structure-aware top result matched the gold page annotation while the fixed-size top result did not. Q07 shows the opposite: fixed-size chunking ranked a relevant page first, while the structure-aware top result was marked incorrect. These examples demonstrate that the preferred strategy depends on the query and the way relevant information is grouped into chunks.

For the five table-based questions, the retrieval logs show that fixed-size chunking ranked the relevant result first for Q21, Q22 and Q23, while structure-aware chunking ranked the relevant result second. Both strategies ranked relevant results first for Q24 and Q25. This is a useful observation, but the five-question table subset is too small to support a broad claim that one chunking method is universally better for tables.

6.2 Main finding

Considering all 25 questions and the four reported metrics, structure-aware chunking is the stronger overall configuration in this experiment. The advantage is clearest in Recall@3 (88% compared with 72%) and Recall@5 (96% compared with 84%). The MRR values are relatively close (0.7743 and 0.7613), indicating a smaller difference in average first-relevant-result rank than the recall differences alone might suggest.

7. Reliability, limitations and future work

The corpus audit and ingestion logs provide evidence that all 30 registered documents were processed successfully for both final collections. Structure-aware ingestion produced 15,374 chunks; fixed-size ingestion produced 19,341 chunks. The fixed-size ingestion log records 30 successful documents and zero failures. The structure-aware final ingestion also completed with 30 successful documents and zero failures. A repeat structure-aware ingestion was idempotent: it added no new chunks and skipped the 30 already processed documents.

7.1 Limitations

The evaluation contains 25 manually specified questions, so the results depend on the coverage and quality of that test set. Recall is based on gold document/page annotations; it does not evaluate answer generation, citation faithfulness or whether a user can fully resolve a real-world regulatory question from the retrieved passage. Table-based questions are represented by five examples only. OCR quality may also vary across scanned documents, and metadata or section detection may be less reliable where the source layout is complex.

7.2 Future improvements

Future work could expand the evaluation set, add more table-focused questions, manually review OCR output for difficult scans, refine section-boundary detection, and evaluate retrieval using additional ranking metrics or human relevance judgments. A later extension could add an LLM answer-generation layer with source-grounded citations, but that would be a separate capability from the retrieval system assessed in this report.

8. Conclusion

PolicyLens implements an end-to-end ingestion and semantic-retrieval workflow for 30 RBI PDF documents. It combines PDF text extraction, OCR fallback, text cleaning, two chunking strategies, metadata enrichment, E5 embeddings and ChromaDB storage. The command-line retrieval interface supports semantic search and metadata filtering, and the evaluation pipeline records ranked results against 25 gold-annotated questions.

In the measured comparison, structure-aware chunking achieved Recall@3 of 88%, Recall@5 of 96%, Recall@10 of 100% and MRR of 0.7743. Fixed-size chunking achieved 72%, 84%, 92% and 0.7613 respectively. Therefore, structure-aware chunking is the preferred configuration for the current PolicyLens test set, while the per-question results show that fixed-size chunking can still be stronger for selected queries.

References and project artifacts

1. Reserve Bank of India. Public RBI circulars, directions and related PDF documents included in the PolicyLens corpus (see **data/manifest.csv** for the document inventory).
2. intfloat. *e5-base-v2* embedding model, used for document and query embeddings.
3. Chroma. Vector database used to store embeddings, text chunks and metadata.
4. Project source code and experiment outputs: **ingest.py**, **search.py**, **src/**, **eval/questions.csv**, **eval/metrics_30docs.csv**, **eval/metrics_fixed_30docs.csv**, and the corresponding retrieval logs.

Before submission: Replace the institution and submission-date placeholders on the cover page. Confirm the required citation style and add any required course, instructor, student ID or declaration details specified by your assignment brief.